

Universidade do Minho
Escola de Engenharia

Cálculo de Programas

Trabalho Prático (2025/26)

Lic. em Ciências da Computação
Lic. em Engenharia Informática

Grupo G99

a107293 Hugo Ferreira Soares
a106902 Francisco Ribeiro Martins

Preâmbulo

Em [Cálculo de Programas](#) pretende-se ensinar a programação de computadores como uma disciplina científica. Para isso parte-se de um repertório de *combinadores* que formam uma álgebra da programação e usam-se esses combinadores para construir programas *composicionalmente*, isto é, agregando programas já existentes.

Na sequência pedagógica dos planos de estudo dos cursos que têm esta disciplina, opta-se pela aplicação deste método à programação em [Haskell](#) (sem prejuízo da sua aplicação a outras linguagens funcionais). Assim, o presente trabalho prático coloca os alunos perante problemas concretos que deverão ser implementados em [Haskell](#). Há ainda um outro objectivo: o de ensinar a documentar programas, a validá-los e a produzir textos técnico-científicos de qualidade.

Antes de abordarem os problemas propostos no trabalho, os grupos devem ler com atenção o anexo [A](#) onde encontrarão as instruções relativas ao *software* a instalar, etc.

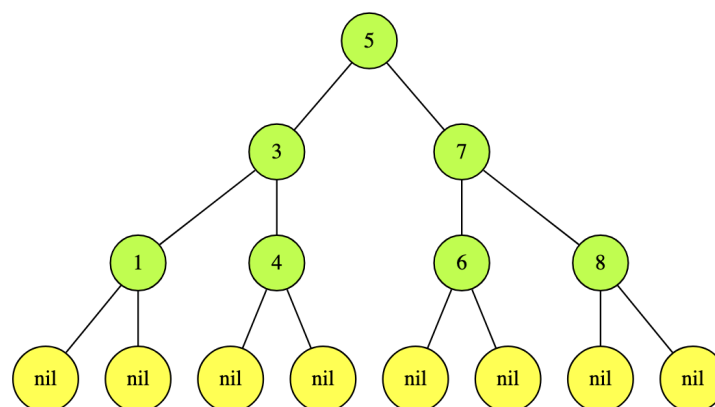
Valoriza-se a escrita de *pouco* código que corresponda a soluções simples e elegantes que utilizem os combinadores de ordem superior estudados na disciplina.

Avaliação. Faz parte da avaliação do trabalho a sua defesa por parte dos elementos de cada grupo. Estes devem estar preparados para responder a perguntas sobre *qualquer* dos problemas deste enunciado. A prestação *individual* de cada aluno nessa defesa oral será uma componente importante e diferenciadora da avaliação.

Problema 1

Uma serialização (ou travessia) de uma árvore é uma sua representação sob a forma de uma lista. Na biblioteca *BTree* encontram-se as funções de serialização *inordt*, *preordt* e *postordt*, que fazem as travessias *in-order*, *pre-order* e *post-order*, respectivamente. Todas essas travessias são catamorfismos que percorrem a árvore argumento em regime *depth-first*.

Pretende-se agora uma função *bfordr* que faça a travessia em regime *breadth-first*, isto é, por níveis. Por exemplo, para a árvore t_1 dada em anexo e mostrada na figura a seguir,



a função deverá dar a lista

[5, 3, 7, 1, 4, 6, 8]

em que se vê como os níveis 5, depois 3, 7 e finalmente 1, 4, 6, 8 foram percorridos.

Pretendemos propor duas versões dessa função:

1. Uma delas envolve um catamorfismo de *BTrees*:

$$\begin{aligned} \text{bfsLevels} &:: \text{BTree } a \rightarrow [a] \\ \text{bfsLevels} &= \text{concat} \cdot \text{levels} \end{aligned}$$

Complete a definição desse catamorfismo:

$$\begin{aligned} \text{levels} &:: \text{BTree } a \rightarrow [[a]] \\ \text{levels} &= \llbracket g\text{levels} \rrbracket \end{aligned}$$

2. A segunda proposta,

$$\text{bft} :: \text{BTree } a \rightarrow [a]$$

deverá basear-se num anamorfismo de listas.

Sugestão: estudar o artigo [2] cujo PDF está incluído no material deste trabalho. Quando fizer testes ao seu código pode, se desejar, usar funções disponíveis na biblioteca *Exp* para visualizar as árvores em GraphViz (formato .dot).

Justifique devidamente a sua resolução, que deverá vir acompanhada de diagramas explicativos. Como já se disse, valoriza-se a escrita de *pouco* código que corresponda a soluções simples e elegantes que utilizem os combinadores de ordem superior estudados na disciplina.

Problema 2

Considere a seguinte função em Haskell:

```
f x = wrapper · worker where
  wrapper = head
  worker 0 = start x
  worker (n + 1) = loop x (worker n)
  loop x [s, h, k, j, m] =
    [h / k + s, x ↑ 2 * h, k * j, j + m, m + 8]
  start x = [x, x ↑ 3, 6, 20, 22]
```

Pode-se provar pela lei de recursividade mútua que $f\ x\ n$ calcula o seno hiperbólico de x , $\sinh x$, para n aproximações da sua série de Taylor. Faça a derivação da função dada a partir da referida série de Taylor, apresentando todos os cálculos justificativos, tal como se faz para outras funções no capítulo respectivo do texto base desta UC [3].

Problema 3

Quem em Braga observar, ao fim da tarde, o tráfego onde a Avenida Clairmont Fernand se junta à N101, aproximadamente na coordenada [41°33'46.8"N 8°24'32.4"W](#) — ver as setas da figura que se segue — reparará nas sequências imparáveis (infinitas!) de veículos provenientes dessas vias de circulação.

Mas também irá observar um comportamento interessante por parte dos condutores desses veículos: por regra, *cada carro numa via deixa passar, à sua frente, exactamente outro carro da outra via*.



Este comportamento *civilizado* chama-se *fair-merge* (ou *fair-interleaving*) de duas sequências infinitas, também designadas *streams* em ciência da computação. Seja dado o tipo dessas sequências em Haskell,

data *Stream* *a* = *Cons* (*a*, *Stream* *a*) **deriving** *Show*

para o qual se define também:

out (*Cons* (*x*, *xs*)) = (*x*, *xs*)

O referido comportamento civilizado pode definir-se, em Haskell, da forma seguinte:¹

```
fair_merge :: (Stream a, Stream a) + (Stream a, Stream a) → Stream a
fair_merge = [h, k] where
  h (Cons (x, xs), y) = Cons (x, k (xs, y))
  k (x, Cons (y, ys)) = Cons (y, h (x, ys))
```

Defina *fair_merge* como um **anamorfismo** de *Streams*, usando o combinador

$\llbracket g \rrbracket = \text{Cons} \cdot (\text{id} \times \llbracket g \rrbracket) \cdot g$

e a seguinte estratégia:

- Derivar a lei **dual** da recursividade mútua,

$$[f, g] = \llbracket [h, k] \rrbracket \equiv \begin{cases} \text{out} \cdot f = F [f, g] \cdot h \\ \text{out} \cdot g = F [f, g] \cdot k \end{cases} \quad (1)$$

tal como se fez, nas aulas, para a que está no formulário.

- Usar (1) na resolução do problema proposto.

Justificar devidamente a resolução, que deverá vir acompanhada de diagramas explicativos.

Problema 4

Como se sabe, é possível pensarmos em catamorfismos, anamorfismos etc *probabilísticos*, quer dizer, programas recursivos que dão distribuições como resultados. Por exemplo, podemos pensar num combinador

pcataList :: (() + (*a*, *b*) → *Dist* *b*) → [*a*] → *Dist* *b*

¹ O facto das sequências serem infinitas não nos deve preocupar, pois em Haskell isso é lidado de forma transparente por [lazy evaluation](#).

que é muito parecido com

$$(\cdot) :: () \rightarrow (a, b) \rightarrow b \rightarrow [a] \rightarrow b$$

da biblioteca [List](#). A principal diferença é que o gene de *pcataList* é uma função probabilística.

Como exemplo de utilização, recorde-se que $(\text{zero}, \text{add})$ soma todos os elementos da lista argumento, por exemplo:

$$(\text{zero}, \text{add}) [20, 10, 5] = 35.$$

Considere-se agora a função *padd* (adição probabilística) que, com probabilidade 90% soma dois números e com probabilidade 10% os subtrai:

$$\text{padd}(a, b) = D[(a + b, 0.9), (a - b, 0.1)]$$

Se se correr

$$d4 = \text{pcataList} [\text{pzero}, \text{padd}] [20, 10, 5] \text{ where } \text{pzero} = \text{return} \cdot \text{zero}$$

obter-se-á:

```
35  81.0%
25   9.0%
 5   9.0%
15   1.0%
```

Com base neste exemplo, resolva o seguinte

Problema: Uma unidade militar pretende enviar uma mensagem urgente a outra, mas tem o aparelho de telegrafia meio avariado. Por experiência, o telegrafista sabe que a probabilidade de uma palavra se perder (não ser transmitida) é 5%; e que, no final de cada mensagem, o aparelho envia o código "stop", mas (por estar meio avariado), falha 10% das vezes.

Qual a probabilidade de a palavra "atacar" da mensagem

`words "Vamos atacar hoje"`

se perder, isto é, o resultado da transmissão ser ["Vamos", "hoje", "stop"]? E a de seguirem todas as palavras, mas faltar o "stop" no fim? E a da transmissão ser perfeita?

Responda a estas perguntas encontrando *gene* tal que

`transmitir = pcataList gene`

descreve o comportamento do aparelho. Justificar devidamente a resolução, que deverá vir acompanhada de diagramas explicativos.

Anexos

A Natureza do trabalho a realizar

Este trabalho teórico-prático deve ser realizado por grupos de 3 alunos. Os detalhes da avaliação (datas para submissão do relatório e sua defesa oral) são os que forem publicados na [página da disciplina](#) na internet.

Recomenda-se uma abordagem participativa dos membros do grupo em **todos** os exercícios do trabalho, para assim poderem responder a qualquer questão colocada na *defesa oral* do relatório.

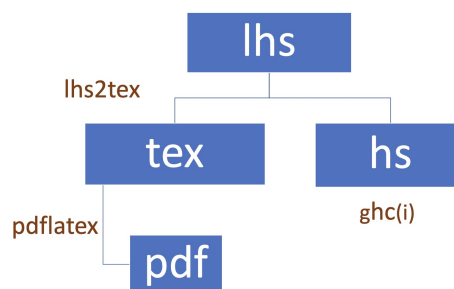
Para cumprir de forma integrada os objectivos do trabalho vamos recorrer a uma técnica de programação dita “literária” [1], cujo princípio base é o seguinte:

Um programa e a sua documentação devem coincidir.

Por outras palavras, o **código fonte** e a **documentação** de um programa deverão estar no mesmo ficheiro.

O ficheiro `cp2526t.pdf` que está a ler é já um exemplo de **programação literária**: foi gerado a partir do texto fonte `cp2526t.lhs`¹ que encontrará no **material pedagógico** desta disciplina descompactando o ficheiro `cp2526t.zip`.

Como se mostra no esquema abaixo, de um único ficheiro (*lhs*) gera-se um PDF ou faz-se a interpretação do código **Haskell** que ele inclui:



Vê-se assim que, para além do **GHCI**, serão necessários os executáveis **pdflatex** e **lhs2TeX**. Para facilitar a instalação e evitar problemas de versões e conflitos com sistemas operativos, é recomendado o uso do **Docker** tal como a seguir se descreve.

B Docker

Recomenda-se o uso do **container** cuja imagem é gerada pelo **Docker** a partir do ficheiro `Dockerfile` que se encontra na diretoria que resulta de descompactar `cp2526t.zip`. Este **container** deverá ser usado na execução do **GHCI** e dos comandos relativos ao **LaTeX**. (Ver também a `Makefile` que é disponibilizada.)

Após **instalar o Docker** e descarregar o referido zip com o código fonte do trabalho, basta executar os seguintes comandos:

```
$ docker build -t cp2526t .  
$ docker run -v ${PWD}:/cp2526t -it cp2526t
```

NB: O objetivo é que o container seja usado *apenas* para executar o **GHCI** e os comandos relativos ao **LaTeX**. Deste modo, é criado um *volume* (cf. a opção `-v ${PWD}:/cp2526t`) que permite que a diretoria em que se encontra na sua máquina local e a diretoria `/cp2526t` no **container** sejam partilhadas.

Pretende-se então que visualize/edite os ficheiros na sua máquina local e que os compile no **container**, executando:

¹ O sufixo ‘lhs’ quer dizer *literate Haskell*.

```
$ lhs2TeX cp2526t.lhs > cp2526t.tex
$ pdflatex cp2526t
```

[lhs2TeX](#) é o pre-processor que faz “pretty printing” de código Haskell em [L^AT_EX](#) e que faz parte já do [container](#). Alternativamente, basta executar

```
$ make
```

para obter o mesmo efeito que acima.

Por outro lado, o mesmo ficheiro `cp2526t.lhs` é executável e contém o “kit” básico, escrito em [Haskell](#), para realizar o trabalho. Basta executar

```
$ ghci cp2526t.lhs
```

Abra o ficheiro `cp2526t.lhs` no seu editor de texto preferido e verifique que assim é: todo o texto que se encontra dentro do ambiente

```
\begin{code}
...
\end{code}
```

é seleccionado pelo [GHCi](#) para ser executado.

C Em que consiste o TP

Em que consiste, então, o *relatório* a que se referiu acima? É a edição do texto que está a ser lido, preenchendo o anexo [G](#) com as respostas. O relatório deverá conter ainda a identificação dos membros do grupo de trabalho, no local respectivo da folha de rosto.

Para gerar o PDF integral do relatório deve-se ainda correr os comando seguintes, que actualizam a bibliografia (com [BibT_EX](#)) e o índice remissivo (com [makeindex](#)),

```
$ bibtex cp2526t.aux
$ makeindex cp2526t.idx
```

e recompilar o texto como acima se indicou. (Como já se disse, pode fazê-lo correndo simplesmente `make` no [container](#).)

No anexo [F](#) disponibiliza-se algum código [Haskell](#) relativo aos problemas que são colocados. Esse anexo deverá ser consultado e analisado à medida que isso for necessário.

Deve ser feito uso da [programação literária](#) para documentar bem o código que se desenvolver, em particular fazendo diagramas explicativos do que foi feito e tal como se explica no anexo [D](#) que se segue.

D Como exprimir cálculos e diagramas em L^AT_EX/lhs2TeX

Como primeiro exemplo, estudar o texto fonte ([lhs](#)) do que está a ler¹ onde se obtém o efeito seguinte:²

$$\begin{aligned} id &= \langle f, g \rangle \\ \equiv \quad &\{ \text{universal property} \} \end{aligned}$$

¹ Procure e.g. por "sec:diagramas".

² Exemplos tirados de [\[3\]](#).

$$\begin{aligned}
& \begin{cases} \pi_1 \cdot id = f \\ \pi_2 \cdot id = g \end{cases} \\
\equiv & \quad \{ \text{identity} \} \\
& \begin{cases} \pi_1 = f \\ \pi_2 = g \end{cases}
\end{aligned}$$

□

Os diagramas podem ser produzidos recorrendo à *package* [xymatrix](#), por exemplo:

$$\begin{array}{ccc}
\mathbb{N}_0 & \xleftarrow{\text{in}} & 1 + \mathbb{N}_0 \\
\downarrow \langle g \rangle & & \downarrow id + \langle g \rangle \\
B & \xleftarrow{g} & 1 + B
\end{array}$$

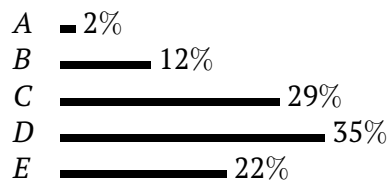
E O mónade das distribuições probabilísticas

Mónades são funtores com propriedades adicionais que nos permitem obter efeitos especiais em programação. Por exemplo, a biblioteca [Probability](#) oferece um mónade para abordar problemas de probabilidades. Nesta biblioteca, o conceito de distribuição estatística é captado pelo tipo

$$\text{newtype Dist } a = D \{ unD :: [(a, ProbRep)] \} \quad (2)$$

em que *ProbRep* é um real de 0 a 1, equivalente a uma escala de 0 a 100%.

Cada par (a, p) numa distribuição $d :: \text{Dist } a$ indica que a probabilidade de a é p , devendo ser garantida a propriedade de que todas as probabilidades de d somam 100%. Por exemplo, a seguinte distribuição de classificações por escalões de A a E ,



será representada pela distribuição

$$\begin{aligned}
d1 &:: \text{Dist Char} \\
d1 &= D [('A', 0.02), ('B', 0.12), ('C', 0.29), ('D', 0.35), ('E', 0.22)]
\end{aligned}$$

que o [GHCi](#) mostrará assim:

```

'D'  35.0%
'C'  29.0%
'E'  22.0%
'B'  12.0%
'A'   2.0%

```

É possível definir geradores de distribuições, por exemplo distribuições *uniformes*,

$$d2 = \text{uniform} (\text{words "Uma frase de cinco palavras"})$$

isto é


```

"Uma"    20.0%
"cinco"  20.0%
"de"     20.0%
"frase"  20.0%
"palavras" 20.0%

```

distribuição *normais*, eg.

```
d3 = normal [10..20]
```

etc.¹ Dist forma um **mónade** cuja unidade é $\text{return } a = D [(a, 1)]$ e cuja composição de Kleisli é (simplificando a notação)

$$(f \bullet g) a = [(y, q * p) \mid (x, p) \leftarrow g a, (y, q) \leftarrow f x]$$

em que $g : A \rightarrow \text{Dist } B$ e $f : B \rightarrow \text{Dist } C$ são funções **monádicas** que representam *computações probabilísticas*.

Este mónade é adequado à resolução de problemas de *probabilidades e estatística* usando programação funcional, de forma elegante e como caso particular da programação monádica.

F Código fornecido

Problema 1

Árvores exemplo:

```

t1 :: BTree Int
t1 = Node (5, (Node (3, (Node (1, (Empty, Empty)), Node (4, (Empty, Empty)))),
  Node (7, (Node (6, (Empty, Empty)), Node (8, (Empty, Empty)))))
t2 :: BTree Int
t2 =
  node 1
    (node 2 (node 4 Empty Empty) (node 5 Empty Empty))
    (node 3 (node 6 Empty Empty) (node 7 Empty Empty))
t3 :: BTree Char
t3 =
  node 'A'
    (node 'B' (node 'C' (node 'D' Empty Empty) Empty) Empty)
    (node 'E' Empty Empty)
t4 :: BTree Char
t4 =
  node 'A'
    (node 'B' (node 'C' (node 'D' Empty Empty) Empty) Empty)
    Empty
t5 :: BTree Int
t5 =
  node 1

```

¹ Para mais detalhes ver o código fonte de [Probability](#), que é uma adaptação da biblioteca [PFP](#) ("Probabilistic Functional Programming"). Para quem quiser saber mais recomenda-se a leitura do artigo [?].

$(node\ 2\ (node\ 4\ Empty\ Empty)\ Empty)$
 $(node\ 3\ Empty\ (node\ 5\ (node\ 6\ Empty\ Empty)\ Empty))$
 $node\ a\ b\ c = Node\ (a,\ (b,\ c))$

G Soluções dos alunos

Os alunos devem colocar neste anexo as suas soluções para os exercícios propostos, de acordo com o “layout” que se fornece. Não podem ser alterados os nomes ou tipos das funções dadas, mas pode ser adicionado texto ao anexo, bem como diagramas e/ou outras funções auxiliares que sejam necessárias.

Importante: Não pode ser alterado o texto deste ficheiro fora deste anexo.

Problema 1

1. Catamorfismo de níveis (*levels*)

A função *levels* visa agrupar os elementos da árvore numa lista de listas, onde cada sublista contém os elementos de um nível. Para tal, definimos o catamorfismo $levels = \langle glevels \rangle$.

O gene *glevels* processa os dois casos do tipo *BTree*:

- No caso *Empty*, retorna uma lista vazia $[]$.
- No caso $Node\ (a,\ (l,\ r))$, onde l e r são as listas de listas já calculadas para as subárvores, o gene coloca a numa lista unitária (nível 0) e utiliza a função *zipWithPlus* para fundir os níveis restantes das duas subárvores.

O diagrama do catamorfismo é o seguinte:

$$\begin{array}{ccc}
 BTree\ a & \xrightarrow{outBTree} & 1 + a \times (BTree\ a \times BTree\ a) \\
 \downarrow levels & & \downarrow id + id \times (levels \times levels) \\
 [[a]] & \xleftarrow{glevels} & 1 + a \times ([[a]] \times [[a]])
 \end{array}$$

$glevels = [[], comb]$

Onde $comb(a, (L, D)) = [a] : zipWith\ (+)\ L\ D$.

- $[a]$ cria o primeiro nível (a raiz).
- $zipWith\ (+)$ junta o nível n da esquerda com o nível n da direita para formar o novo nível $n+1$.

2. Anamorfismo de travessia (*bft*)

Para a função *bft*, utilizamos a estratégia sugerida por Okasaki, baseada numa fila (*queue*). Esta função é um anamorfismo de listas onde o estado é uma lista de árvores $[BTree\ a]$.

O gene *geneBFT* funciona da seguinte forma:

- Se a fila estiver vazia $[]$, a travessia termina.
- Se o primeiro elemento for *Empty*, ignoramos e continuamos com o resto da fila.

- Se for um *Node* $(a, (l, r))$, extraímos o valor a e adicionamos as subárvores l e r ao

fim

da fila (comportamento FIFO), garantindo a ordem por níveis.

```
glevels = [[], λ(a, (ls, rs)) → [a] : zipWithPlus ls rs]
where zipWithPlus [] ys = ys
      zipWithPlus xs [] = xs
      zipWithPlus (x : xs) (y : ys) = (x ++ y) : zipWithPlus xs ys
bft t = [geneBFT] [t]
where geneBFT [] = i1 ()
      geneBFT (Empty : ts) = geneBFT ts
      geneBFT (Node (a, (l, r)) : ts) = i2 (a, ts ++ [l, r])
```

Problema 2

Resolução Teórica

A função $f \times n$ implementa o cálculo do seno hiperbólico através da sua série de Taylor:

$$\sinh(x) = \sum_{i=0}^{\infty} a_i \quad \text{onde} \quad a_i = \frac{x^{2i+1}}{(2i+1)!}$$

A eficiência desta implementação reside na relação de recorrência entre termos consecutivos, evitando o cálculo redundante de fatoriais e potências:

$$a_i = \frac{h_{i-1}}{k_{i-1}} \quad \text{com} \quad h_i = x^2 \cdot h_{i-1} \quad \text{e} \quad k_i = k_{i-1} \cdot j_{i-1}$$

O catamorfismo de números naturais (*worker*) evolui conforme o seguinte diagrama:

$$\begin{array}{ccc} \mathbb{N}_0 & \xrightarrow{\text{outNat}} & 1 + \mathbb{N}_0 \\ \text{worker} \downarrow & & \downarrow \text{id} + \text{worker} \\ [\text{Double}] & \xleftarrow[\text{start } x, \text{loop } x]{} & 1 + [\text{Double}] \end{array}$$

Análise das Componentes do Estado:

Para $n = 0$, o estado inicial é $\text{start } x = [x, x \uparrow 3, 6, 20, 22]$. A lógica dos acumuladores é a seguinte:

- s : Soma acumulada. Começa com x (o termo a_0).
- h : Numerador do próximo termo. Começa em x^3 (para a_1).
- k : Denominador do próximo termo. Começa em $3! = 6$ (para a_1).
- j, m : Auxiliares para a progressão do denominador. j representa o multiplicador para o próximo fatorial ímpar. A constante m (que aumenta de 8 em 8) garante que j siga a sequência $(2n + 2)(2n + 3)$.

O *wrapper* (função *head*) extrai o primeiro elemento da lista, que é a soma acumulada final s após n iterações.

Implementação

```

f :: Double → Int → Double
f x n = wrapper (worker x n)
  where
    wrapper = head
    worker x = ([start x, loop x])
    start x = [x, x ↑ 3, 6, 20, 22]
    loop x [s, h, k, j, m] = [s + h / k, x ↑ 2 * h, k * j, j + m, m + 8]
sumList :: Num a ⇒ [a] → a
sumList = ([0, (+)])
productList :: Num a ⇒ [a] → a
productList = ([1, (*)])
lengthList :: [a] → Int
lengthList = ([0, succ · π₂])

```

Problema 3

Lei de Fokkinga Dual

Para Streams (listas infinitas), o anamorfismo é definido pelo gene $g : S \rightarrow A \times S$. A Lei de Fokkinga Dual permite-nos fundir dois anamorfismos que partilham a mesma estrutura de saída, simplificando a composição de sistemas dinâmicos. A sua derivação baseia-se na propriedade de que o produto de dois anamorfismos é equivalente a um anamorfismo cujo gene é o produto dos genes originais, garantindo a comutatividade do diagrama.

Diagrama do Anamorfismo

O *fairMerge* alterna entre duas streams. O estado do anamorfismo é o par de streams $(Stream\ a, Stream\ a)$ e, em cada passo, o sistema retira um elemento de uma e passa o controlo para a outra (trocando a ordem no par).

$$\begin{array}{ccc}
 Stream\ a & \xrightarrow{\text{out}} & a \times Stream\ a \\
 \uparrow \text{fairMerge} & & \uparrow id \times \text{fairMerge} \\
 (Stream\ a, Stream\ a) & \xrightarrow{\text{gene}} & a \times (Stream\ a, Stream\ a)
 \end{array}$$

Análise do Gene: O gene do *fairMerge* recebe duas streams $(s1, s2)$. Ele utiliza o *out* na primeira para obter a cabeça h e a cauda t , e devolve o par $(h, (s2, t))$. Note-se a inversão de $s2$ com t , que é o que garante a alternância entre as estradas (Braga e Porto) em cada iteração.

Implementação

```

fairMerge :: (Stream a, Stream a) → Stream a
fairMerge = ([gene])
  where
    gene (s1, s2) = let (h, t) = out s1

```

```

    in (h, (s2, t))
fairMergeM :: (Double, (Stream a, Stream a)) → Stream a
fairMergeM = [(geneM)]
where
  geneM (p, (s1, s2)) =
    let r = 0.5
    in if p > r
      then let (h, t) = out s1 in (h, (p, (s2, t)))
      else let (h, t) = out s2 in (h, (p, (s1, t)))

```

Problema 4

Justificação Teórica: Máquinas de Mealy como Anamorfismos

As Máquinas de Mealy são transdutores de estados que modelam sistemas reativos, onde a saída depende tanto do estado atual como da entrada recebida. Diferente de um sistema fechado, uma Máquina de Mealy é inerentemente coindutiva, sendo modelada como um anamorfismo que produz uma sequência potencialmente infinita de respostas a estímulos externos.

Formalmente, o comportamento de uma máquina com estados em S , entradas em I e saídas em O é definido pelo par de funções:

- $out : S \times I \rightarrow O$ (produção de output)
- $next : S \times I \rightarrow S$ (transição de estado)

Pela lei da exponenciação (currying), este par de funções é isomorfo a uma única função de transição, o gene do anamorfismo: $g : S \rightarrow (I \rightarrow O \times S)$. Este gene mapeia o estado atual para uma função que, dada uma entrada, determina o resultado imediato e o próximo estado da máquina.

Diagrama do Anamorfismo (Mealy)

O diagrama seguinte ilustra a estrutura da máquina. Note-se que o tipo de saída do anamorfismo é ele próprio uma função ($I \rightarrow O \times \dots$), caracterizando a natureza interativa do sistema:

$$\begin{array}{ccc}
 S & \xrightarrow{g} & I \rightarrow O \times S \\
 \text{mealy} \downarrow & & \downarrow \text{id} \rightarrow \text{id} \times \text{mealy} \\
 \text{Mealy } I \text{ } O & \xleftarrow{\text{inMealy}} & I \rightarrow O \times \text{Mealy } I \text{ } O
 \end{array}$$

Implementação: Fluxo de Tráfego em Braga

No contexto da rede viária, modelamos uma avenida onde o estado é a cor do semáforo ($Bool$). O gene g dita que, se o semáforo estiver verde ($True$), a entrada de carros é integralmente transferida para a saída. Se estiver vermelho ($False$), a saída é uma lista vazia, simulando a retenção do tráfego. Em cada passo, o estado alterna, garantindo a fluidez alternada do sistema.

```

avenidaMealy :: Bool → Mealy [Carro] [Carro]
avenidaMealy = anaMealy gene
where
  gene verde entrada =

```

```
if verde
then (entrada,  $\neg$  verde)
else ([],  $\neg$  verde)
transitoBraga :: Mealy [Carro] [Carro]
transitoBraga = avenidaMealy True · avenidaMealy False
```

Index

$\LaTeX$, [5](#), [6](#)

bibtex, [6](#)

lhs2TeX, [5](#), [6](#)

makeindex, [6](#)

pdflatex, [5](#)

xymatrix, [7](#)

Combinador “pointfree”

ana, [3](#)

cata

 Naturais, [7](#)

either, [3](#), [4](#)

split, [6](#)

Cálculo de Programas, [1](#), [4](#)

 Material Pedagógico, [5](#)

 List.hs, [4](#)

Docker, [5](#)

 container, [5](#), [6](#)

Functor, [3](#), [7](#), [8](#)

Função

π_1 , [7](#)

π_2 , [7](#)

Haskell, [1](#), [5](#), [6](#)

 Biblioteca

 PFP, [8](#)

 Probability, [7](#), [8](#)

 interpretador

 GHCi, [5–7](#)

 Lazy evaluation, [3](#)

 Literate Haskell, [5](#)

Números naturais ($\mathbb{N}$), [7](#)

Programação

 literária, [5](#), [6](#)

References

- [1] D.E. Knuth. *Literate Programming*. CSLI Lecture Notes Number 27. Stanford University Center for the Study of Language and Information, Stanford, CA, USA, 1992.
- [2] Chris Okasaki. Breadth-first numbering: lessons from a small exercise in algorithm design. In Martin Odersky and Philip Wadler, editors, *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00), Montreal, Canada, September 18-21, 2000*, pages 131–136. ACM, 2000.
- [3] J.N. Oliveira. Program Design by Calculation, 2024. Draft of textbook in preparation. First version: 1998. Current version: Sep. 2024. Informatics Department, University of Minho ([pdf](#)).